

Normalized and denormalized data

So far, you've learned about normalization and denormalization. As a refresher, normalization is the process of structuring a relational database in accordance with a series of so-called normal forms to reduce data redundancy, and improve data integrity. Denormalization is a database optimization technique in which redundant data is added to one or more tables. In this reading, you'll learn more about normalization and denormalization, when it's most useful to normalize or denormalize data, and what storage systems these different data formats require.

When to normalize or denormalize data

There are advantages to both normalization and denormalization that can help you decide what kind of data is appropriate for your organization's data needs.

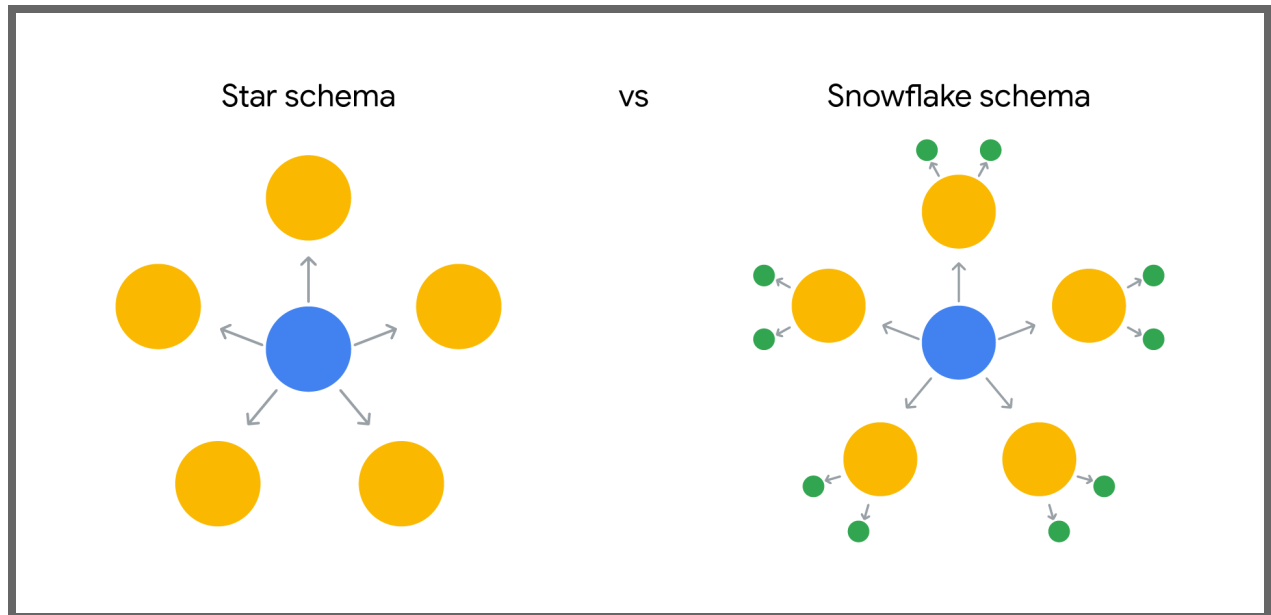
Data format	Advantages
Normalized	• Avoids duplicate data and inconsistencies • Quickly applies updates • Optimizes memory to promote performance • Maintains integrity
Denormalized	• Improves execution time for queries • Reduces number of tables to cut down on storage resources • Is an alternative to expensive table JOINS • Enhances scalability for changing data needs • Reduces overall complexity

Depending on your organization's needs, you might choose to normalize or denormalize your database. For example, in a transactional database, read speed and fast performance are necessary to quickly fetch information, and complete transactions. But, if there isn't a need to JOIN tables very often, you might choose to normalize your database to promote these functions. On the other hand, for analytical systems where data needs to be consistent and clean for analysis, and users are making updates or JOINING tables frequently, a denormalized system is more beneficial because the tables can be pre-joined.

How normalized and denormalized data is stored

Traditionally, normalized data is organized using relational database schemas, like star and snowflake models. These schemas use shared fields called database keys to connect tables, and build relationships between them that allow users to JOIN them. An example of a relational

schema is a star schema, which consists of one fact table that references any number of dimension tables. Similarly, a snowflake schema is an extension of a star schema with additional dimensions and, often, subdimensions.



These schemas can be denormalized by adding duplicate columns, splitting tables, and creating mirror tables that copy the original, or even summary tables.

There are also completely denormalized schemas, like flat models. Flattened schemas are simple database systems with a single table in which each record is represented by a single row of data. Flat models are not relational; they don't depend on additional relationships between tables. Instead, the single table already models all possible relationships, which can result in duplicated data. Because of this, flat models are usually used as a potential source within a data system to capture less complex data that doesn't need to be updated, like historical data used for analysis in a data warehouse.

Key takeaways

As a cloud data professional, recognizing how an organization is using data, and what their needs are, will help you develop a cloud system. Normalized and denormalized data both have distinct advantages you may want to consider. So, recognizing the utility of these different data formats can help you design and implement the right system for your organization.